

Introduction

This mini project is a simple Banking System made using Python.

The system allows a single user to:

- Deposit money into an account.
- Withdraw money from the account with basic checks.
- See the current balance at any time.

2. Problem Statement and Approach

2.1 Problem Statement

In real life, a bank account supports some basic operations like deposit, withdraw, and check balance.

The problem is:

- Design a system where the user can create an account (in memory), deposit and withdraw money, and check the balance using a simple menu.

2.2 Approach

The approach used in this project is:

- Use OOP (Object-Oriented Programming):
 - Create a **BankAccount** class to represent one bank account.
 - Put data (account holder name, balance) and functions (deposit, withdraw, check_balance) inside this class.
- Use a menu-driven program:
 - In the **main()** function, display options (Deposit, Withdraw, Check Balance, Exit).
 - Take user input in a loop so the user can perform multiple transactions until they choose Exit.
- Use basic validation:
 - Do not allow negative or zero deposit/withdraw amounts.
 - Do not allow withdrawing more money than the current balance.

This approach keeps the project simple, understandable, and good enough for a college-level demonstration.

3. Implementation Details

3.1 Code Structure

Main components of the program:

1. **BankAccount** class

- Attributes:
 - ♦ **account_holder**: stores the name of the account holder.
 - ♦ **balance**: stores the current account balance (default is 0.0).
- Methods:
 - ♦ **__init__(self, account_holder, balance=0.0)**:

- ◊ Constructor, called when a new **BankAccount** object is created.
- ◊ Sets the holder name and starting balance.
- ◆ **deposit(self, amount):**
 - ◊ Checks that **amount** is greater than 0.
 - ◊ Adds **amount** to **self.balance**.
 - ◊ Prints the deposited amount and the new balance.
- ◆ **withdraw(self, amount):**
 - ◊ Checks that **amount** is greater than 0.
 - ◊ If **amount** is more than current **self.balance**, shows "Insufficient balance."
 - ◊ Otherwise subtracts **amount** from **self.balance** and prints the new balance.
- ◆ **check_balance(self):**
 - ◊ Prints the current balance.

2. **main()** function

- Prints a welcome message.
- Asks the user to enter the account holder name.
- Creates a **BankAccount** object with that name.
- Runs a **while True** loop that:
 - ◆ Shows a menu:
 1. Deposit Money
 2. Withdraw Money
 3. Check Balance
 4. Exit
 - ◆ Takes the user's choice and calls the appropriate method on the **BankAccount** object.
 - ◆ Breaks the loop when the user selects 4 (Exit).

3. **if __name__ == "__main__":** block

- Ensures that **main()** runs only when this file is executed directly.

3.2 Flow of Execution (Step-by-Step)

1. Program starts and executes **main()**.
2. User enters their name; a **BankAccount** object is created with balance 0.
3. Menu is shown:
 - If user chooses 1:
 - ◆ Program asks for an amount.
 - ◆ **deposit()** is called and balance increases.
 - If user chooses 2:
 - ◆ Program asks for an amount.
 - ◆ **withdraw()** is called and balance decreases if enough money is available.
 - If user chooses 3:
 - ◆ **check_balance()** is called and current balance is printed.
 - If user chooses 4:

- ♦ Program prints a thank-you message and exits the loop.
- Any other input: program shows an error message and shows the menu again.

4. Results (Sample Runs)

Here are some example scenarios that show the working of the system. You can put these as "Result Screenshots" by actually running the code and taking screenshots.

4.1 Scenario 1: Simple Deposit

- User name: "Amit".
- Starting balance: 0.
- User selects "1. Deposit Money" and enters 1000.
- Output shows:
 - Amount deposited.
 - New balance: 1000.

This shows that the deposit function is working correctly and adding to the balance.

4.2 Scenario 2: Valid Withdrawal

- Current balance: 1000.
- User selects "2. Withdraw Money" and enters 400.
- Output shows:
 - Amount withdrawn.
 - New balance: 600.

This shows that withdrawal reduces the balance correctly when there is enough money.

4.3 Scenario 3: Insufficient Balance

- Current balance: 600.
- User tries to withdraw 1000.
- Output shows "Insufficient balance." and balance stays 600.

This shows that the program correctly blocks withdrawals larger than the balance.

4.4 Scenario 4: Invalid Amount

- User tries to deposit or withdraw 0 or a negative number.
- Output shows a message like "Deposit amount must be positive." or "Withdrawal amount must be positive."

This shows basic input validation for the transaction amount.

5. Analysis and Discussion

5.1 Strengths of the Project

- Simple and easy to understand:
The logic is straightforward, so other students and teachers can read and understand the code easily.
- Uses OOP concepts:

The **BankAccount** class shows how to combine data and methods, which is a key OOP idea.

- Handles basic validation:

The program checks for invalid amounts and insufficient balance, which makes it more realistic than a very raw example.

- Good for learning:

The project helps in practicing:

- Classes and objects
- Functions and loops
- Conditional statements
- User input and output

5.2 Limitations

- Single account only:

The current version supports only one account at a time. In a real bank system, there would be many accounts.

- No data storage:

When the program closes, all information is lost because data is not stored in a file or database.

- No security:

There is no login system, password, or PIN. Anyone using the program can access the account.

- No interest or extra features:

Real banking systems have many more operations like interest calculation, account types, transaction history, etc., which are not include.

These enhancements would make the project closer to a real banking application and suitable for higher-level courses.

Time Complexity

Consider each operation on the **BankAccount** object:

- **deposit(amount)**

- Only does a few constant-time actions: a check on **amount**, one addition, and a print.
- Time complexity: $O(1)$
 $O(1)$ per deposit operation.

- **withdraw(amount)**

- Also does a few constant-time actions: checks on **amount** and **balance**, one subtraction, and a print.
- Time complexity: $O(1)$
 $O(1)$ per withdraw operation.

- **check_balance()**

- Just reads the balance and prints it.
- Time complexity: $O(1)$.
- Menu loop in **main()**
 - Each iteration of the loop performs exactly one of the above operations, which are all $O(1)$.
 - If the user performs n operations before exiting, total time = $n \times O(1) = O(n)$.

So, per single user action (one choice in the menu), the time complexity is $O(1)$.

Space Complexity

- The **BankAccount** object stores:
 - **account_holder** (a string).
 - **balance** (a float).
 - A few local variables in **main()** (like **choice**, **amount**).
- All of these take a fixed amount of memory that does not grow with the number of operations.
- Therefore, space complexity of the current program (single account, no lists or arrays) is: $O(1)$.

If in future you extend it to handle multiple accounts in a list or dictionary, then space complexity would change to

$O(m)$

$O(m)$ where

m

m is the number of accounts store

6. Conclusion

This Banking System mini project demonstrates how basic banking operations can be modeled in Python using classes and simple console input/output. Although it is not a full real-world system, it gives a clear idea of how larger banking applications can be built step by step.